



Programming Mobile Applications in Flutter

Communication with Native

When do I need to use platform-specific APIs

- Notifications, app lifecycle, deep links
- Sensors, camera, battery, geolocation, sound, connectivity
- Sharing information with other apps, launching other apps
- Persisted preferences, special folders, device information
- Intensive, high performance requiring tasks (e.g. Bitmap processing)
- ...

How Flutter handles native APIs?

Flutter approach to native APIs

- Only core APIs are part of the framework (pointer events, keyboard etc.)
- Everything else is provided by **plugins***

shared_preferences 2.2.2

Published 2 months ago •  [flutter.dev](https://pub.dev/packages/shared_preferences) Dart 3 compatible

geolocator 10.1.0 

Published 3 months ago •  [baseflow.com](https://pub.dev/packages/geolocator) Dart 3 compatible

flutter_secure_storage 9.0.0

Published 4 months ago •  [steenbakker.dev](https://pub.dev/packages/flutter_secure_storage) Dart 3 compatible

What is a plugin?

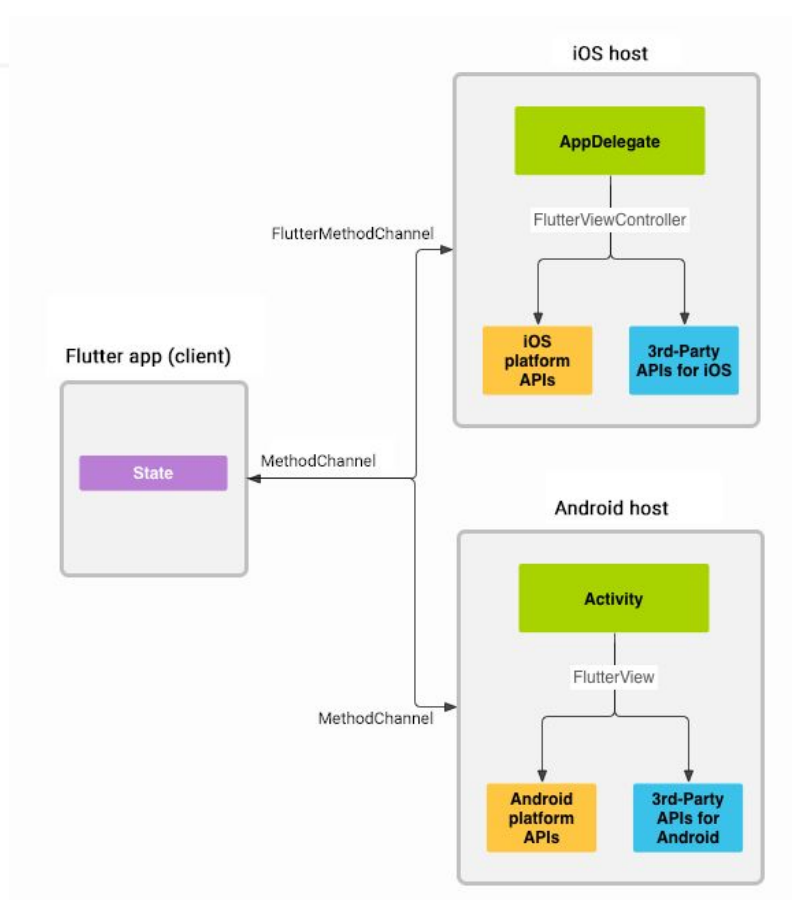
A specialized Dart package that contains an API written in Dart code combined with one or more platform-specific implementations.

How plugins communicate with native APIs?

- Method channels
- JavaScript interoperability
- C interop

Method channels

A named channel for communicating with platform plugins using asynchronous method calls. Method calls are encoded into binary before being sent, and binary results received are decoded into Dart values.



Source: <https://docs.flutter.dev/development/platform-integration/platform-channels>

Which language can I use?

Flutter uses a flexible system that allows you to call platform-specific APIs in a language that works directly with those APIs:

- Kotlin or Java on Android
- Swift or Objective-C on iOS
- C++ on Windows
- Swift or Objective-C on macOS
- C on Linux

```
class _MyHomePageState extends State<MyHomePage> {
  static const platform = MethodChannel('samples.flutter.dev/battery');

  String _batteryLevel = 'Unknown battery level.';

  Future<void> _getBatteryLevel() async {
    String batteryLevel;
    try {
      final int result = await platform.invokeMethod('getBatteryLevel');
      batteryLevel = 'Battery level at $result % .';
    } on PlatformException catch (e) {
      batteryLevel = "Failed to get battery level: '${e.message}'.";
    }

    setState(() {
      _batteryLevel = batteryLevel;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Material(
      child: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            ElevatedButton(
              child: const Text('Get Battery Level'),
              onPressed: _getBatteryLevel,
            ), // ElevatedButton
            Text(_batteryLevel),
          ],
        ), // Column
      ), // Center
    ); // Material
  }
}
```

```
class MainActivity: FlutterActivity() {  
    private val CHANNEL = "samples.flutter.dev/battery"  
  
    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {  
        super.configureFlutterEngine(flutterEngine)  
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL).setMethodCallHandler {  
            call, result ->  
            // Note: this method is invoked on the main thread.  
            // TODO  
        }  
    }  
}
```

```
@UIApplicationMain
@objc class AppDelegate: FlutterAppDelegate {
  override func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {

    let controller : FlutterViewController = window?.rootViewController as! FlutterViewController
    let batteryChannel = FlutterMethodChannel(name: "samples.flutter.dev/battery",
                                              binaryMessenger: controller.binaryMessenger)

    batteryChannel.setMethodCallHandler({
      (call: FlutterMethodCall, result: @escaping FlutterResult) -> Void in
        // Note: this method is invoked on the UI thread.
        // Handle battery messages.
    })

    GeneratedPluginRegistrant.register(with: self)
    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
  }
}
```

```

override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
    super.configureFlutterEngine(flutterEngine)
    MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL).setMethodCallHandler {
        // Note: this method is invoked on the main thread.
        call, result ->
        if (call.method == "getBatteryLevel") {
            val batteryLevel = getBatteryLevel()

            if (batteryLevel != -1) {
                result.success(batteryLevel)
            } else {
                result.error( errorCode: "UNAVAILABLE",  errorMessage: "Battery level not available.",  errorDetails: null)
            }
        } else {
            result.notImplemented()
        }
    }
}

private fun getBatteryLevel(): Int {
    return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
        val batteryManager = getSystemService(Context.BATTERY_SERVICE) as BatteryManager
        batteryManager.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPACITY)
    } else {
        val intent = ContextWrapper(applicationContext).registerReceiver(
            receiver: null, IntentFilter(
                Intent.ACTION_BATTERY_CHANGED
            )
        )
        intent!!.getIntExtra(BatteryManager.EXTRA_LEVEL, defaultValue: -1) * 100 / intent.getIntExtra(
            BatteryManager.EXTRA_SCALE,
            defaultValue: -1
        )
    }
}

```

```
final int result = await platform.invokeMethod(  
    'getBatteryLevel',  
    {  
        "currentBatteryLevel": _batteryLevel,  
    },  
);
```

```
if (call.method == "getBatteryLevel") {  
    val currentBatteryLevel = call.argument<String>(key: "currentBatteryLevel")
```

Dart and platform types

Dart	Kotlin
null	null
bool	Boolean
int	Int
int, if 32 bits not enough	Long
double	Double
String	String
Uint8List	ByteArray
Int32List	IntArray
Int64List	LongArray
Float32List	FloatArray
Float64List	DoubleArray
List	List
Map	HashMap

What if I need to react to some platform events?

Event channels

A named channel for communicating with platform plugins using event streams!

```
static const eventChannel = EventChannel("sample.flutter/randomValueEventChannel");

@override
void initState() {
  eventChannel.receiveBroadcastStream().distinct().listen((event) {
    setState(() {
      _randomValue = event;
    });
  });
  super.initState();
}
```

```
private fun setupEventChannel(flutterEngine: FlutterEngine) {
    EventChannel(
        flutterEngine.dartExecutor.binaryMessenger,
        name: "sample.flutter/randomValueEventChannel"
    )
        .setStreamHandler(object : EventChannel.StreamHandler {
            override fun onListen(arguments: Any?, events: EventChannel.EventSink) {
                thread {
                    while (true) {
                        runOnUiThread { events.success(Random.nextInt()) }
                        Thread.sleep( millis: 2000)
                    }
                }
            }

            override fun onCancel(arguments: Any?) = Unit
        })
}
```

Contracts for MethodChannels

Pigeon

```
class Value {  
    int? number;  
}  
  
@HostApi()  
abstract class Api2Host {  
    @async  
    Value calculate(Value value);  
}
```

// Swift

```
/** Generated interface from Pigeon that represents a handler of messages from Flutter.*/  
protocol Api2Host {  
    func calculate(value: Value, completion: @escaping (Value) -> Void)  
}
```

// Kotlin

```
/** Generated interface from Pigeon that represents a handler of messages from Flutter.*/  
interface Api2Host {  
    fun calculate(value: Value, callback: (Value) -> Unit)  
}
```

C interop

- Allows calling functions written in C
- Can be used synchronously
- Requires memory management
- Objective-C, Swift, Java, and Kotlin interop
 - jnigen for JVM
 - ffi for ObjC/Swift

```
void hello_world()  
{  
    printf("Hello World\n");  
}
```

```
// FFI signature of the hello_world C function
typedef HelloWorldFunc = ffi.Void Function();
// Dart type definition for calling the C foreign function
typedef HelloWorld = void Function();
```

Run | Debug

```
void main() {
  // Open the dynamic library
  var libraryPath =
    path.join(Directory.current.path, 'hello_library', 'libhello.so');

  if (Platform.isMacOS) {
    libraryPath =
      path.join(Directory.current.path, 'hello_library', 'libhello.dylib');
  }

  if (Platform.isWindows) {
    libraryPath = path.join(
      Directory.current.path, 'hello_library', 'Debug', 'hello.dll');
  }

  final dylib = ffi.DynamicLibrary.open(libraryPath);

  // Look up the C function 'hello_world'
  final HelloWorld hello = dylib
    .lookup<ffi.NativeFunction<HelloWorldFunc>>('hello_world')
    .asFunction();
  // Call the function
  hello();
}
```


Old JavaScript interoperability

- Available only on web
- Can be called synchronously

```
@JS()  
library stringify;  
  
import 'package:js/js.dart';  
  
// Calls invoke JavaScript `JSON.stringify(obj)`.  
@JS('JSON.stringify')  
external String stringify(Object obj);
```

New JavaScript/WASM interoperability

The following table maps Dart's new JS and web interop solutions to their past counterparts:

New interop libraries

`package:web`

`dart:js_interop`
`dart:js_interop_unsafe`

Previous libraries

`dart:html`
`dart:indexed_db`
`dart:svg`
`dart:web_audio`
`dart:web_gl`

`package:js`
`dart:js`
`dart:js_util`

Top-level interop members

Given the following JS members:

```
globalThis.name = 'global';  
globalThis.isEmpty = function() {  
  return globalThis.name.length == 0;  
}
```

js

You can write interop members for them like so:

```
@JS()  
external String get name;  
  
@JS()  
external set name(String value);  
  
@JS()  
external bool isEmpty();
```

dart

dart

```
import 'dart:js_interop';

@JS()
external set exportedFunction(JSFunction value);

void printString(JSString string) {
  print(string.toDart);
}

void main() {
  exportedFunction = printString.toJS;
}
```

js

```
globalThis.exportedFunction('hello world');
```

Can I use native views?

Platform View

- **AndroidView** - Embeds an Android view in the Widget hierarchy
- **UIKitView** - Embeds an iOS view in the Widget hierarchy
- **HtmlElementView** - Embeds an HTML element in the Widget hierarchy in Flutter Web
- ...

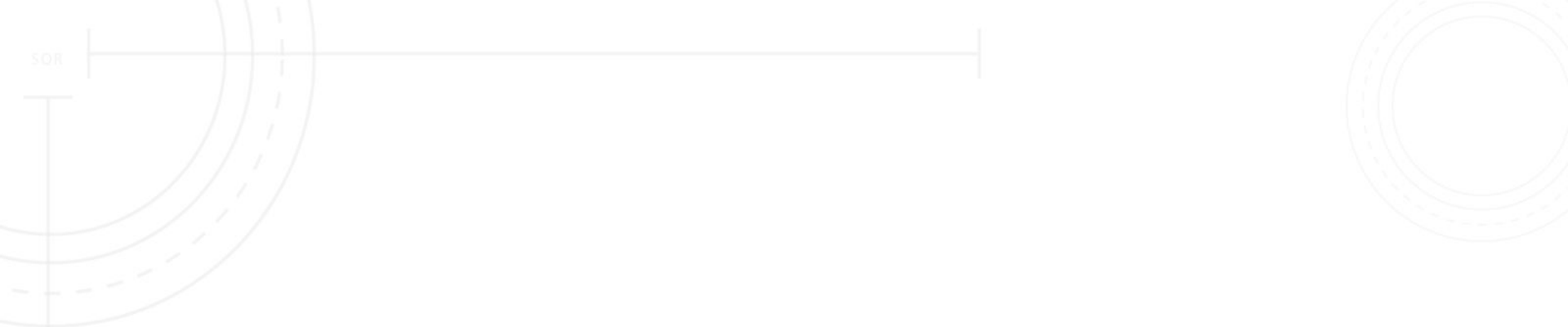
```
internal class NativeView(context: Context, id: Int, creationParams: Map<String?, Any?>?) : PlatformView {
    private val textView by lazy {
        TextView(context).apply { this: TextView
            textSize = 24f
            text = "Rendered on a native Android view (id: $id)"
        }
    }

    override fun getView(): View {
        return textView
    }

    override fun dispose() = Unit
}

class NativeViewFactory : PlatformViewFactory(StandardMessageCodec.INSTANCE) {
    override fun create(context: Context, viewId: Int, args: Any?): PlatformView {
        val creationParams = args as Map<String?, Any?>
        return NativeView(context, viewId, creationParams)
    }
}
```

```
flutterEngine
    .platformViewsController
    .registry
    .registerViewFactory( viewType: "androidNativeViewType", NativeViewFactory())
```

```
— AndroidView(  
    viewType: "androidNativeViewType",  
), // AndroidView
```

Should I know Native if want to be a Flutter Developer?

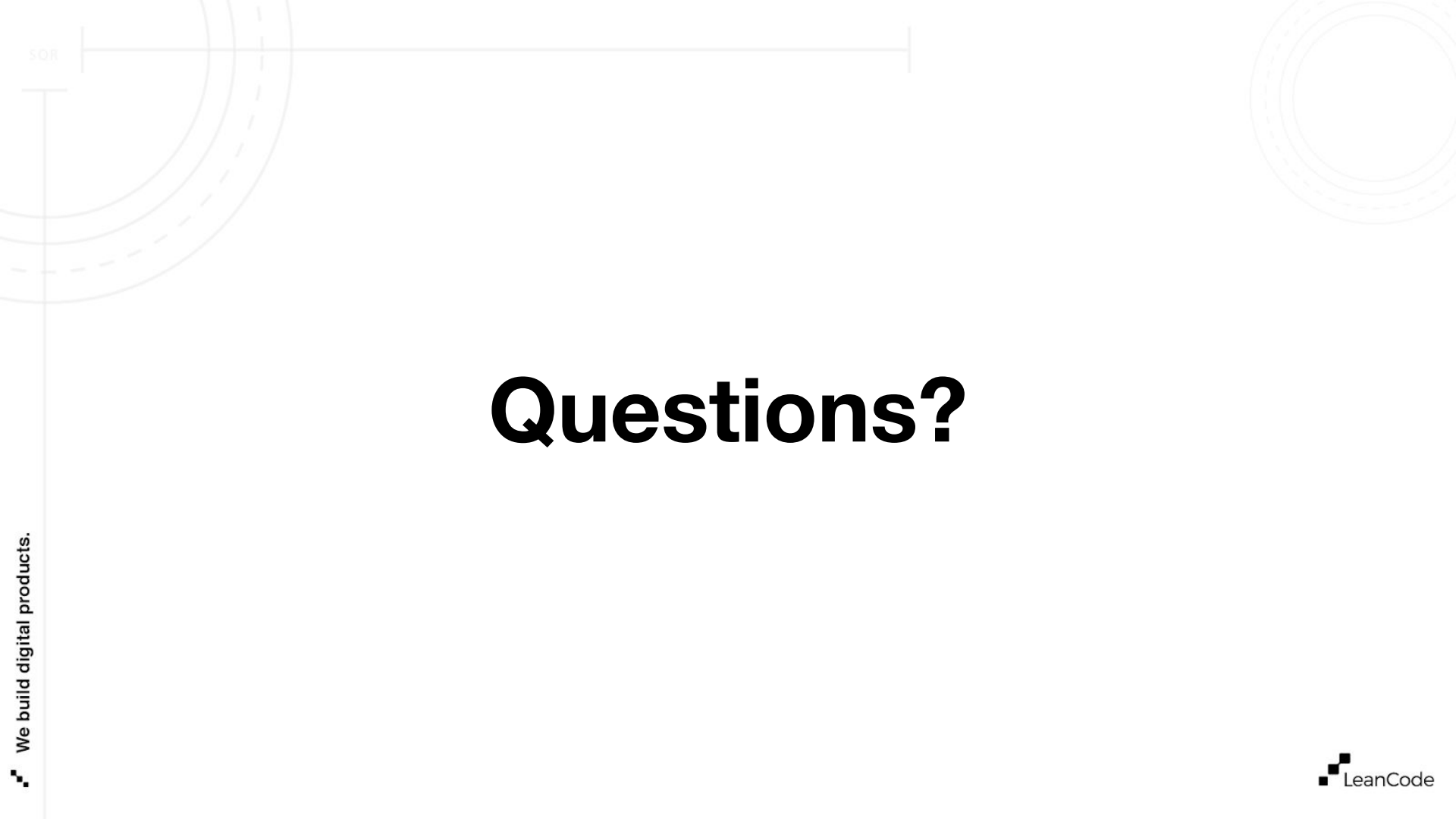
Yes*

Should I know Native?

- Flutter is a good starting point for learning Native
- Most of the time you won't have to touch native code due to big number of available plugins
- Native platforms knowledge gives you more possibilities and flexibility as you can easily integrate every Android/iOS library and do certain optimizations

Can I embed Flutter in an existing Native App?

**Of course! You can even
embed Flutter in React Native
app**



Questions?